

AI-Based Food Recognition with Nutrition-Aware Recommendations

Jaspreet Singh*, Inderpal Singh†

*† Department of Computer Science and Engineering
National Institute of Technology Srinagar
Kashmir 190006, India
Email: [student email]

Abstract—Food-recognition applications are useful only when the predicted food label is connected to reliable nutrition information. This paper presents an AI-based Indian food recognition system that detects visible food items, links them to nutrition records, calculates macronutrients, and generates nutrition-aware recommendations from the computed values. The main technical contribution is a conservative nutrition-grounding layer placed between the YOLO detector and the recommendation module. Instead of relying on direct fuzzy string matching, the layer uses canonical class names, explicit semantic mappings, source-backed supplemental rows, thresholded fallback matching, and provenance metadata. Five YOLO-format Indian-food datasets were merged into a 72-class dataset with 48,693 exported image-label pairs. A YOLO11s detector trained on this dataset achieved 0.820 mAP@50 and 0.680 mAP@50:95 on the held-out test split. The nutrition database contains 1,010 food records and 103 detector-to-database mappings, covering all 72 expanded classes. Macro verification reported 51 verified mappings, 21 review-level mappings, and zero failed mappings. The resulting prototype connects image upload, detection overlays, nutrition lookup, serving adjustment, and personalized recommendations while keeping the numerical nutrition layer separate from generated advice.

Index Terms—food recognition, nutrition-aware recommendations, YOLO, Indian food dataset, dietary assessment, macronutrients

I. INTRODUCTION

Most food-recognition systems stop at the visual task: they identify what appears in an image. A diet-assistance application has to go one step further. A detected class such as `palak_paneer`, `rice`, or `samosa` must be linked to a nutrition entry before the system can say anything useful about calories, protein, carbohydrates, or fat. If this link is wrong, the final recommendation may look precise while being nutritionally misleading.

This problem is especially visible in Indian meals. The same dish can appear under different spellings, regional names, or dataset-specific labels. For example, plain rice may be listed as `WhiteRice`, `satham`, or `rice`. A chutney label may refer to the side dish itself or to a small portion beside a larger food item. Biryani and pulao classes may look similar in an image but differ in ingredients and macro values. These cases make direct string matching a weak basis for nutrition lookup.

This paper presents a project titled *AI-Based Food Recognition with Nutrition-Aware Recommendations*. The system uses YOLO11s for object detection, a curated Indian-food

dataset, a SQLite nutrition database, and a recommendation layer that receives already-computed nutrition values. The key idea is to keep recognition, nutrition grounding, and recommendation generation separate. The detector predicts visible food items; the nutrition layer decides whether a safe database match exists; the recommendation module uses those values to produce diet-aware guidance.

The contributions are:

- 1) a 72-class Indian-food object-detection dataset built by merging and normalizing five YOLO-format datasets;
- 2) a conservative detector-to-nutrition mapping method that reduces unsafe nutrition substitutions;
- 3) a working FastAPI and Next.js prototype that connects detection, macro calculation, serving adjustment, and personalized recommendations.

II. RELATED WORK

The literature reviewed for this project falls into three groups: Indian food recognition, nutrition database alignment, and recommendation systems. Work outside this scope, especially 3D volume or depth-based portion estimation, is not central to this paper because the implemented system reports database-backed per-100g macro values and leaves portion adjustment to the user.

Indian food recognition work shows why a domain-specific detector is needed. IndianFoodNet uses YOLO-style detection for Indian food items, which is closer to real meal images than single-label classification because one plate may contain several dishes [1]. Khana presents a larger Indian cuisine classification benchmark and highlights the familiar problems of class imbalance, visual similarity, and regional variation [2]. The Indian Thali study goes further toward cluttered meal analysis through segmentation and controlled capture, but that setup is less convenient for a normal web-upload workflow [3]. Lightweight CNN and MobileNetV3 studies also support the use of compact models and train-time augmentation, though they remain closer to classification than multi-item detection [4], [5].

The nutrition side of the literature is equally important. The Indian Food Composition Database work explains why cooked Indian recipes need dedicated nutrient records rather than direct raw-ingredient assumptions [6]. The Synergistic Frameworks study directly identifies the compatibility problem

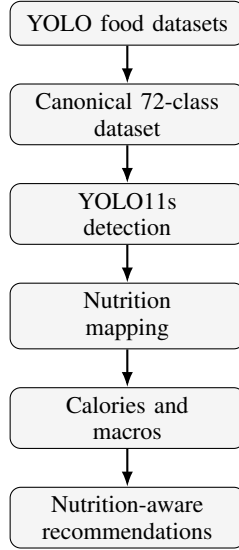


Fig. 1. End-to-end workflow for AI-based food recognition with nutrition-aware recommendations.

between visual food labels and nutrition databases: the detector may use one label, while the nutrition row uses a different spelling, language, or preparation name [7]. FKG.in shows how language models and food knowledge graphs can help parse regional recipe descriptions, but the present system begins from a photograph rather than a typed recipe [8]. This is why the implemented backend uses deterministic database lookup and audited class-to-food mappings instead of asking a language model to invent nutrition values.

Recent recommendation systems show the value and the risk of generative AI in diet applications. NutriGen and other web-based meal-planning systems use LLMs to generate personalized plans from structured user constraints [9], [10]. Diet Engine combines food detection with NLP-based dietary guidance, which is close to the high-level direction of this project [11]. At the same time, evaluations of ChatGPT-style nutrient estimation from meal photographs warn that fluent responses do not guarantee accurate calorie or macro values [12]. The design used here follows that lesson: the recommendation layer receives detected foods and calculated macros from the backend, while nutrition arithmetic remains database-backed and inspectable.

III. PROPOSED SYSTEM

Fig. 1 summarizes the implemented system. The pipeline starts with dataset normalization and model training. At runtime, the frontend sends an uploaded food image to the backend. The backend runs YOLO11s, resolves each predicted class through the nutrition mapping table, calculates macro values, and returns the structured result to the frontend. Recommendation text is generated only after the numeric nutrition values have been calculated.

Let a detector return

$$D_x = \{(\hat{c}_q, b_q, p_q)\}_{q=1}^{Q_x}, \quad (1)$$

where $\hat{c}_q$ is the predicted food class, b_q is the bounding box, and p_q is the confidence score. Let the nutrition database be

$$N = \{(n_j, \mathbf{m}_j, z_j)\}_{j=1}^L, \quad (2)$$

where n_j is a food name, $\mathbf{m}_j = [e_j, \text{protein}_j, \text{carb}_j, \text{fat}_j]$ is the nutrition vector, and z_j stores source metadata.

The mapping function M resolves a detector class to a nutrition row:

$$M(c) = \begin{cases} M_{\text{manual}}(c), & \text{if a reviewed semantic match exists,} \\ M_{\text{exact}}(c), & \text{if normalized names match,} \\ M_{\text{sup}}(c), & \text{if a verified supplemental row exists,} \\ M_{\text{fuzzy}}(c), & \text{if } \text{score}(c, n) \geq 0.78, \\ \emptyset, & \text{otherwise.} \end{cases} \quad (3)$$

The final case is important. If no safe nutrition match exists, the system keeps the detection but does not invent macro values. In a health-oriented interface, a missing value is preferable to a wrong confident value.

For a meal image, the displayed macro vector is

$$\hat{\mathbf{m}}(x) = \sum_{q=1}^{Q_x} s_q \mathbf{m}_{M(\hat{c}_q)}, \quad (4)$$

where s_q is the serving multiplier selected by the user. This keeps portion adjustment explicit rather than pretending that exact food weight can be recovered from a single image.

A. Runtime Data Contract

The backend response is designed so that the frontend does not need to infer the meaning of missing or approximate values. Each detection carries the raw detector label, display name, confidence score, bounding box, image dimensions, macro values, nutrition unit, nutrition source, and missing-nutrition flag. If no valid food is detected, the API returns an explicit no-food state. If a class is detected but no safe nutrition match exists, the detection remains visible while macro values are left unavailable. This choice makes the system easier to inspect and prevents a recommendation from being based on hidden guesses.

IV. DATASET CONSTRUCTION

Five YOLO-format Indian-food datasets were merged into one canonical dataset [13]. Source class names were decoded from each dataset configuration file, normalized for case and punctuation, and then reviewed for semantic variants. Examples include mapping `AllooGobi`, `aloo gobhi`, and `aloo_gobi` to `aloo_gobi`; mapping `WhiteRice`, `satham`, and `rice` to `rice`; and mapping `medu vadai` to `medu_vada`. Multi-food label files were preserved by rewriting every valid YOLO annotation row.

The merge process used a 70/15/15 target ratio for training, validation, and test splits. Multi-label images were retained because a realistic food photograph may contain several dishes. Augmentation was applied only to the training split, mainly for underrepresented classes, so that validation and test metrics were not inflated by synthetic variants of the same images.

TABLE I
SOURCE DATASET POOL BEFORE PROJECT-SPECIFIC MERGING.

Source dataset	Classes	Train	Valid	Test
Indian food detection v1	17	7547	1053	453
Indian food v6	29	8088	213	105
Indian food v2 subset	7	1698	159	92
IndianFoodNet YOLO export	30	11387	1073	576
South Indian food detection v19	31	6478	1294	581

TABLE II
REPRESENTATIVE GROUPS IN THE 72-CLASS INDIAN-FOOD LABEL SPACE.

Group	Example classes
Rice dishes	rice, lemon_rice, sambar_satham, veg_briyani
Breads	roti, paratha, bhakri, bhatura, puri
Curries	chole, rajma_curry, fish_curry, palak_paneer
Snacks	samosa, onion_pakoda, medu_vada, khandvi
Sweets	gulab_jamun, ras_malai, kheer, jalebi
Sides	coconut_chutney, green_chutney, raita, salad

TABLE III
GENERATED DATASET SPLIT SUMMARY.

Item	Result
Canonical food classes	72
Train image-label pairs	36,852
Validation image-label pairs	5,922
Held-out test image-label pairs	5,919
Total exported pairs	48,693
Classes with visual samples	72 / 72
Known annotation issues found	3

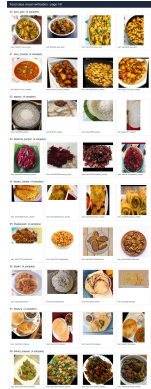


Fig. 2. Sample visual verification sheet generated from real exported dataset images.

A visual verification script sampled real images for every class and generated contact sheets. This check confirmed that all 72 classes have image samples. It also found three local annotation issues: one jalebi sample that did not visually match jalebi, one kaara_chutney box placed on a vada, and one extremely thin nandu_masala box. These issues were recorded for future cleanup and did not indicate a class-order error.

V. DETECTOR TRAINING AND EVALUATION

The final detector was trained with YOLO11s at 640-pixel image size and batch size 16. Training resumed from epoch 56 and continued to epoch 100 on a Tesla T4 GPU. The best logged validation mAP@50 was 0.83379 at epoch 78. On the held-out test split, the exported model reached 0.842 precision, 0.801 recall, 0.820 mAP@50, and 0.680 mAP@50:95 across 5,919 images and 8,875 instances.

TABLE IV
FINAL DETECTOR EVALUATION ON VALIDATION AND HELD-OUT TEST SPLITS.

Split	Precision	Recall	mAP@50	mAP@50:95
Validation	0.847	0.807	0.830	0.692
Held-out test	0.842	0.801	0.820	0.680

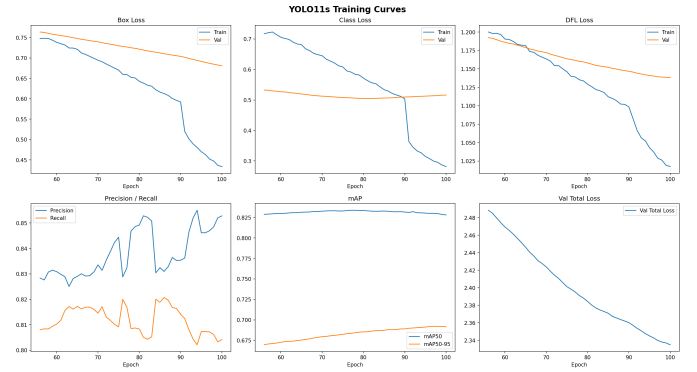


Fig. 3. YOLO11s training curves for the 72-class Indian-food detector.

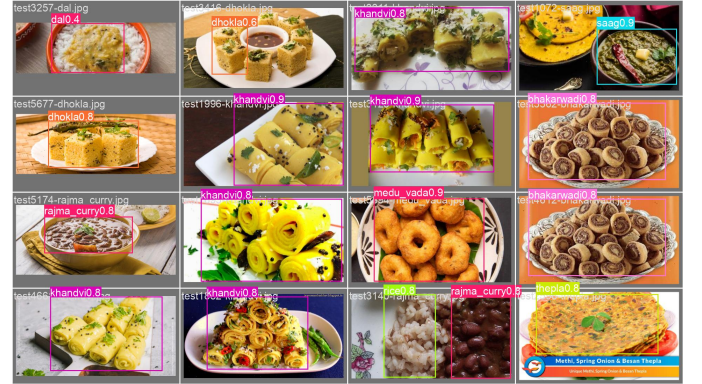


Fig. 4. Example held-out test predictions from the final YOLO11s detector.

The small validation-to-test gap is important for this project because the detector is only one part of the user-facing system. A model that performs well only on validation data would make the later nutrition and recommendation layers appear stronger than they are. The held-out result suggests that the detector generalizes reasonably across the generated split, although several visually similar categories still need deeper per-class error analysis.

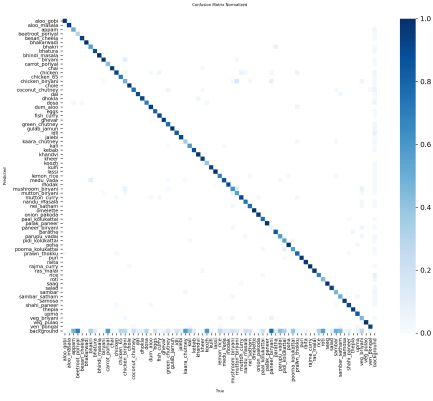


Fig. 5. Normalized confusion matrix from held-out test evaluation.

VI. NUTRITION MAPPING AND RECOMMENDATIONS

The nutrition database is built from an Indian Nutrient Database spreadsheet and stored in SQLite [14]. The final table contains 1,010 food records with calories, protein, carbohydrates, fat, and source metadata. Five foods absent from the available INDB sheet were added as verified supplemental rows: bhakarwadi, ghevar, jalebi, khandvi, and nandu_masala [15], [16], [17], [18], [19].

TABLE V
REPRESENTATIVE DETECTOR-TO-NUTRITION MAPPINGS.

Detector class	Nutrition database row	Score
aloo_gobi	Potato cauliflower (Aloo gobi)	1.00
palak_paneer	Spinach paneer (Palak paneer)	1.00
rice	Boiled rice (Uble chawal)	1.00
chicken_biryani	Chicken pulao	0.80
ven_pongali	Plain khitchdi	0.75

Table V shows that not all mappings are equally strong. Direct semantic matches are treated as verified. Lower-score preparation-family matches are kept available but marked for review. This distinction matters because a recommendation should be based on known nutrition values, not on hidden assumptions.

TABLE VI
NUTRITION MAPPING VERIFICATION.

Validation item	Result
Nutrition food records	1,010
YOLO-to-database mappings	103
Expanded classes checked	72
Expanded classes with mappings	72
Verified mappings	51
Review mappings with complete macros	21
Failed macro mappings	0
Expanded-class coverage	100.00%

The recommendation module uses the detected foods, calculated calories, macro totals, user goal, calorie target, and health profile. It does not generate the calorie and macronutrient

values itself. This separation keeps the advice layer from becoming a source of fabricated numeric nutrition data.

The macro engine converts a calorie target into carbohydrate, protein, and fat grams. A selected goal defines the base macro split, and a health profile modifies that split before normalization. For a calorie target T and final macro percentages p_c , p_p , and p_f , the target grams are:

$$G_c = \frac{Tp_c}{4 \times 100}, \quad G_p = \frac{Tp_p}{4 \times 100}, \quad G_f = \frac{Tp_f}{9 \times 100}. \quad (5)$$

The recommendation service receives these calculated values along with detected foods. For example, a high-carbohydrate fried meal under a weight-loss goal can trigger portion-control and protein-pairing advice, while a diabetic profile can shift the emphasis toward carbohydrate moderation and fiber.

TABLE VII
BASE MACRO SPLITS USED BEFORE HEALTH-PROFILE ADJUSTMENT.

Goal	Carbs	Protein	Fat
Weight loss	40%	35%	25%
Muscle gain	40%	30%	30%
Maintenance	50%	25%	25%
Endurance	55%	20%	20%

VII. PROTOTYPE VALIDATION

The backend is implemented with FastAPI, PyTorch, Ultralytics YOLO, SQLite, and supporting Python libraries. The frontend is built with Next.js, React, TypeScript, and Tailwind CSS. A user can upload an image, inspect detected boxes, review nutrition cards, adjust serving multipliers, and request dietary suggestions.

TABLE VIII
MAIN IMPLEMENTATION COMPONENTS AND RESPONSIBILITIES.

Component	Responsibility
Dataset builder	Parses source YOLO labels, normalizes class names, exports canonical splits
Vision service	Loads YOLO weights, runs inference, filters boxes, formats detections
Nutrition service	Loads SQLite rows, applies mappings, returns macros and provenance
Macro engine	Converts calorie targets and health profiles into macro goals
Recommendation service	Generates advice from detected foods and computed macros
Frontend interface	Handles upload, overlays, serving controls, macro display, and reset flow

The image-analysis response includes the detector label, display name, confidence, bounding box, image dimensions, macro values, nutrition source, and missing-nutrition flags. Runtime checks were run for classes that are easy to mis-handle, including aloo_gobi, palak_paneer, rice, AlooGobi, and WhiteRice. These resolved to the expected nutrition rows. The API also returns an explicit no-food state when no valid detection is found.

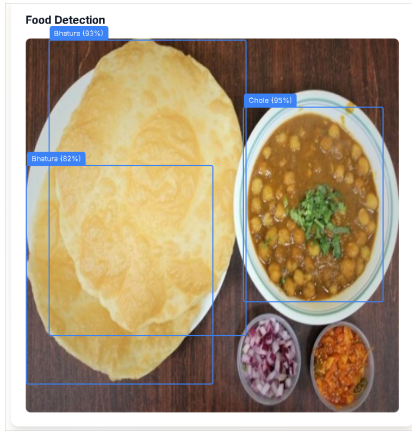


Fig. 6. Prototype output showing detected foods with bounding boxes and nutrition-linked results.

TABLE IX
RUNTIME BEHAVIOR CHECKED DURING PROTOTYPE VALIDATION.

Condition	Expected behavior
aloo_gobi	Resolve to potato cauliflower nutrition row
palak_paneer	Resolve to spinach paneer nutrition row
WhiteRice	Normalize legacy label to boiled rice
No detected food	Return empty detections and no-food flag
Unmapped class	Preserve detection and mark nutrition unavailable

VIII. VALIDATION SCOPE

The project was validated at several levels because a user-facing nutrition result depends on more than one model score. Dataset checks confirmed that class folders, label files, split counts, and visual samples were present. Model evaluation measured detection performance on validation and held-out test splits. Nutrition checks verified that detector classes resolve to complete macro rows. Runtime checks then confirmed that the backend response contains the fields the frontend needs for overlays, nutrition display, and recommendation prompts.

TABLE X
VALIDATION LAYERS USED IN THE PROJECT.

Layer	What was checked
Dataset	Split counts, class coverage, visual samples, annotation issues
Detector	Precision, recall, mAP@50, mAP@50:95, prediction examples
Nutrition	Database rows, mapping coverage, macro completeness, sources
API	Upload validation, no-food state, mapped and unmapped detections
Frontend	Overlay rendering, nutrition cards, serving controls, recommendations

This layered validation is useful for peer review because each failure mode is different. A detector can classify a food

correctly while the nutrition service maps it poorly. A nutrition database can be complete while the frontend receives fields under unexpected names. Treating these checks separately makes the system easier to debug and makes the reported results more transparent.

IX. DISCUSSION

The project shows that the most fragile part of a nutrition-aware food assistant is often not the detector itself. A visually correct prediction can still become a poor dietary result if it is mapped to the wrong food-composition row. This is why the mapping table, source metadata, and explicit missing-value behavior are treated as part of the AI pipeline rather than as backend convenience code.

The system also shows a practical trade-off. It does not attempt exact portion estimation from a single image, because that would require assumptions that are difficult to verify in ordinary user photographs. Instead, it reports database-backed macro values and lets the user adjust serving multipliers. This is less ambitious than automatic plate-weight estimation, but it is more defensible for a working prototype.

X. LIMITATIONS

The system does not estimate exact portion mass from the image. Nutrition values are reported from database rows and adjusted through user-selected serving multipliers. Better portion estimation would need segmentation, depth cues, reference objects, or calibrated serving-size assumptions. The detector also has uneven per-class behavior for visually similar foods and classes with fewer diverse samples. Finally, the 21 review mappings should be interpreted as practical approximations rather than exact recipe equivalence.

XI. CONCLUSION

This paper presented *AI-Based Food Recognition with Nutrition-Aware Recommendations*, an end-to-end system for Indian food detection, nutrition lookup, macro calculation, and personalized dietary guidance. The project combines a YOLO11s detector, a 72-class canonical Indian-food dataset, a conservative nutrition-mapping layer, and a full-stack prototype. The detector achieved 0.820 mAP@50 on the held-out test split, while the nutrition layer mapped all 72 expanded classes with zero failed macro mappings. The main lesson from the project is straightforward: food recognition becomes useful for dietary support only when visual predictions are grounded in auditable nutrition data and passed to the recommendation layer with their uncertainty preserved.

ACKNOWLEDGMENT

The implementation and dataset engineering artifacts used in this manuscript were developed as part of a final year project in the Department of Computer Science and Engineering, National Institute of Technology Srinagar.

REFERENCES

- [1] R. Agarwal, T. Choudhury, N. J. Ahuja, and T. Sarkar, "IndianFoodNet: Detecting indian food items using deep learning," *International Journal of Computational Methods and Experimental Measurements*, vol. 11, no. 4, pp. 221–232, 2023.
- [2] O. Prabhu, "Khana: A comprehensive Indian cuisine dataset," *arXiv preprint arXiv:2509.06006*, 2025.
- [3] Y. Arora, A. Arun, and C. V. Jawahar, "What is there in an Indian Thali?" in *Proceedings of the 16th Indian Conference on Computer Vision, Graphics and Image Processing*, 2025.
- [4] R. U. Haque, R. H. Khan, A. S. M. Shihavuddin, M. M. M. Syeed, and M. F. Uddin, "Lightweight and parameter-optimized real-time food calorie estimation from images using CNN-based approach," *Applied Sciences*, vol. 12, no. 19, p. 9733, 2022.
- [5] J. Patel and K. Modi, "Indian food image classification and recognition with transfer learning technique using MobileNetV3 and data augmentation," in *Engineering Proceedings*, vol. 56, no. 1, 2023, p. 197.
- [6] A. Vijayakumar, H. B. Dubasi, A. Awasthi, and L. M. Jaacks, "Development of an Indian food composition database," *Current Developments in Nutrition*, vol. 8, no. 7, p. 103790, 2024.
- [7] "Synergistic frameworks for Indian food computing: An analysis of nutritional and visual dataset compatibility," 2025.
- [8] S. K. Gupta, L. Dey, P. P. Das, G. Trilok-Kumar, and R. Jain, "Enhancing FKG.in: Automating Indian food composition analysis," *arXiv preprint arXiv:2412.05248*, 2024.
- [9] S. Khamesian, A. Arefeen, S. M. Carpenter, and H. Ghasemzadeh, "NutriGen: Personalized meal plan generator leveraging large language models to enhance dietary and nutritional adherence," *arXiv preprint arXiv:2502.20601*, 2025.
- [10] S. Hussain, N. Khan, S. A. A. Shah, S. Bano, and A. W. Khan, "AI-driven personalized meal planning: A web-based platform for tailored nutrition and health management," *International Journal of Computer Applications*, vol. 187, no. 57, pp. 17–29, 2025.
- [11] A. M. Saad, M. R. H. Rahi, M. M. Islam, and G. Rabbani, "Diet engine: A real-time food nutrition assistant system for personalized dietary guidance," *Food Chemistry Advances*, vol. 7, p. 100978, 2025.
- [12] C. O'Hara, G. Kent, A. C. Flynn, E. R. Gibney, and C. M. Timon, "An evaluation of ChatGPT for nutrient content estimation from meal photographs," *Nutrients*, vol. 17, no. 4, p. 607, 2025.
- [13] Roboflow Dataset Contributors, "Yolo-format indian food datasets used for project dataset engineering," 2026, pLACEHOLDER BIBTEX ENTRY: replace with exact source dataset URLs and access dates before final submission.
- [14] Indian Nutrient Database, "Indian nutrient database spreadsheet used in project data layer," 2026, pLACEHOLDER BIBTEX ENTRY: replace with the official publication or institutional citation for INDB.xlsx before final submission.
- [15] FatSecret India, "Evolve bhakarwadi nutrition facts per 100 g," Online nutrition record, 2026, accessed 2026-06-06.
- [16] Clearcalcs, "Ghevar recipe calories and nutrition per 100 g," Online nutrition record, 2026, accessed 2026-06-06.
- [17] —, "Jalebi recipe calories and nutrition per 100 g," Online nutrition record, 2026, accessed 2026-06-06.
- [18] —, "Khandvi recipe calories and nutrition per 100 g," Online nutrition record, 2026, accessed 2026-06-06.
- [19] —, "Nandu kari recipe calories and nutrition per 100 g," Online nutrition record, 2026, accessed 2026-06-06.